

28. preflight 判定ロジック

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/solar_preflight_logic.py
種別	Python
区分	runtime helper
役割	計測鮮度や command gate の純判定を Node 本体から切り出したロジック関数群。
起動文脈	preflight と speed bridge の共通判定層。

このファイルを読む理由

計測鮮度や command gate の純判定を Node 本体から切り出したロジック関数群。

- evaluate_freshness と evaluate_command_gate が中心。

呼び出し関係

どこから呼ばれるか

- mpc_solarcar/solar_preflight_node.py
- mpc_solarcar/speed_command_bridge_node.py

次に読むと理解が進むファイル

- 特記事項なし。

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
@dataclass(frozen=True)
class FreshnessResult:
    state: str
    health: float
    diagnostic: str

@dataclass(frozen=True)
class CommandGateResult:
    allowed: bool
    reason: str

def evaluate_freshness(
    *,
    elapsed_sec: float,
    ages_sec: dict[str, float | None],
    required: tuple[str, ...],
    timeout_sec: float,
    startup_grace_sec: float,
```

主要構造の読み取り

主要クラスは FreshnessResult, CommandGateResult。主要関数は evaluate_freshness, evaluate_command_gate。

主要クラス・関数

行	種別	名前	補足
7	class	FreshnessResult	
14	class	CommandGateResult	
19	function	evaluate_freshness	
43	function	evaluate_command_gate	

ファイルを上から読んだときの定義順

行	内容
7	クラス FreshnessResult を定義する。
14	クラス CommandGateResult を定義する。
19	関数 evaluate_freshness を定義する。
43	関数 evaluate_command_gate を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないが、間接利用である。
3	<code>from dataclasses import dataclass</code>	<code>dataclasses</code> から <code>dataclass</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L6, L13。

関数・クラスを上から順に解説

L7 クラス `FreshnessResult`

- 定義: `FreshnessResult(bases=)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `dataclass`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `state` にを代入する。
 2. `health` にを代入する。
 3. `diagnostic` にを代入する。

L14 クラス `CommandGateResult`

- 定義: `CommandGateResult(bases=)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `dataclass`
 - 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
1. `allowed` にを代入する。
 2. `reason` にを代入する。

L19 関数 `evaluate_freshness`

- 定義: `evaluate_freshness(elapsed_sec, ages_sec, required, timeout_sec, startup_grace_sec)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `FreshnessResult`, `float`, `get`, `join`
 - 戻り値の要点: `FreshnessResult('RUNNING', 1.0, 'solar telemetry and planner inputs are fresh')` / `FreshnessResult('STARTING', 0.25, 'waiting for required solar telemetry')` / `FreshnessResult('DEGRADED', 0.2, 'missing: ' + ', '.join(missing))` / `FreshnessResult('DEGRADED', 0.4, 'stale: ' + ', '.join(stale))`
1. 条件 `elapsed_sec < startup_grace_sec` を判定し、真なら内部処理を行う。
 2. `missing` に `[name for name in required if ages_sec.get(name) is None]` の結果を代入する。
 3. `stale` に `[name for name in required if ages_sec.get(name) is not None and float(ages_sec[name]) > timeout_sec]` の結果を代入する。

4. 条件 `missing` を判定し、真なら内部処理を行う。
5. 条件 `stale` を判定し、真なら内部処理を行う。
6. `FreshnessResult('RUNNING', 1.0, 'solar telemetry and planner inputs are fresh')` を返す。

L43 関数 `evaluate_command_gate`

- 定義: `evaluate_command_gate(elapsed_sec, speed_input_age_sec, system_state, system_state_age_sec, startup_hold_sec, input_timeout_sec, system_state_timeout_sec, require_system_running)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `CommandGateResult`, `max`, `str`, `strip`, `upper`
 - 戻り値の要点: `CommandGateResult(True, 'ok')` / `CommandGateResult(False, 'startup_hold')` / `CommandGateResult(False, 'missing_speed_command')` / `CommandGateResult(False, 'stale_speed_command')`
1. 条件 `elapsed_sec < max(0.0, startup_hold_sec)` を判定し、真なら内部処理を行う。
 2. 条件 `speed_input_age_sec is None` を判定し、真なら内部処理を行う。
 3. 条件 `speed_input_age_sec > max(0.0, input_timeout_sec)` を判定し、真なら内部処理を行う。
 4. 条件 `not require_system_running` を判定し、真なら内部処理を行う。
 5. 条件 `system_state_age_sec is None` を判定し、真なら内部処理を行う。
 6. 条件 `system_state_age_sec > max(0.0, system_state_timeout_sec)` を判定し、真なら内部処理を行う。
 7. 条件 `str(system_state).strip().upper() != 'RUNNING'` を判定し、真なら内部処理を行う。
 8. `CommandGateResult(True, 'ok')` を返す。

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_preflight_logic.py` を **runtime helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。